LIVRABLE OFFICIEL

Mission 3

Le Remboursement

LSP · ISP · Clean Code

Information	Détail
Équipe	Groupe CCE11
Rédacteurs	Youssef El Harizi · Boudour Zlaoui · Khouloud Fares
Projet ciblé	AutoLoc API (Spring Boot / Java 17)
Principe appliqué	Liskov Substitution Principle + Interface Segregation Principle
Date de rendu	Octobre 2026

Sommaire

1 **Le principe de conception appliqué** LSP & ISP

2 **Schéma de la solution** UML & séquence

3 **Proposition de code détaillée** 6 classes / interfaces

4 **Prompts de la phase de découverte** Esprit critique

> **En une phrase**
> Séparer la capacité d'encaisser de celle de rembourser : seuls les modes qui savent rembourser promettent de le faire. Résultat : plus aucune exception piégée, plus aucun instanceof.

1 Le principe de conception appliqué

1.1 Nom du principe

LSP — Liskov Substitution Principle

Principe de Substitution de Liskov

En synergie avec le principe **ISP — Interface Segregation Principle** (Ségrégation des Interfaces), qui en constitue la solution technique d'implémentation.

1.2 Définition dans nos propres mots

« Si une classe S dérive ou implémente une interface T, alors tout programme manipulant des objets de type T doit pouvoir utiliser des objets de type S sans subir d'anomalie, de crash imprévu, ni d'affaiblissement des garanties du contrat de base. Autrement dit : un sous-type ne doit jamais déclarer savoir faire quelque chose qu'il est incapable d'honorer à l'exécution. »

1.3 Problème mis en évidence et conformité du code

A. Le problème dans LocationManager et la proposition initiale du stagiaire

Dans le code spaghetti original de LocationManager (bloc 4), les modes de paiement étaient traités par une série de 'if-else' rigides. Pour introduire le remboursement lors de l'annulation d'une réservation, un stagiaire a proposé l'interface suivante :

```
public interface IModePaiement {
    Paiement payer(double montant);
    void rembourser(Paiement paiement); // imposé arbitrairement à tous les modes !
}

public class PaiementEspeces implements IModePaiement {
    public Paiement payer(double montant) { /* encaissement */ }
    public void rembourser(Paiement paiement) {
        throw new UnsupportedOperationException("impossible"); // VIOLATION MAJEURE DE LSP
    }
}
```

Pourquoi cette conception est dangereuse en entreprise

1. **Rupture du contrat polymorphique :** lorsqu'un service appelle mode.rembourser(p), il fait confiance au contrat IModePaiement. Si le mode utilisé est du cash, le serveur crashe avec une UnsupportedOperationException en production.
2. **Pollution du code par 'instanceof' :** pour éviter le crash, les développeurs seraient tentés d'ajouter des 'if (mode instanceof PaiementEspeces)' partout dans les contrôleurs, ce qui viole également le principe OCP (Open/Closed).
3. **L'analogie classique :** forcer PaiementEspeces à implémenter rembourser() en levant une exception équivaut au problème du CanardEnPlastique qui hérite de Canard avec une méthode voler() qui explose.

B. Notre solution Clean Code (Avant vs Après)

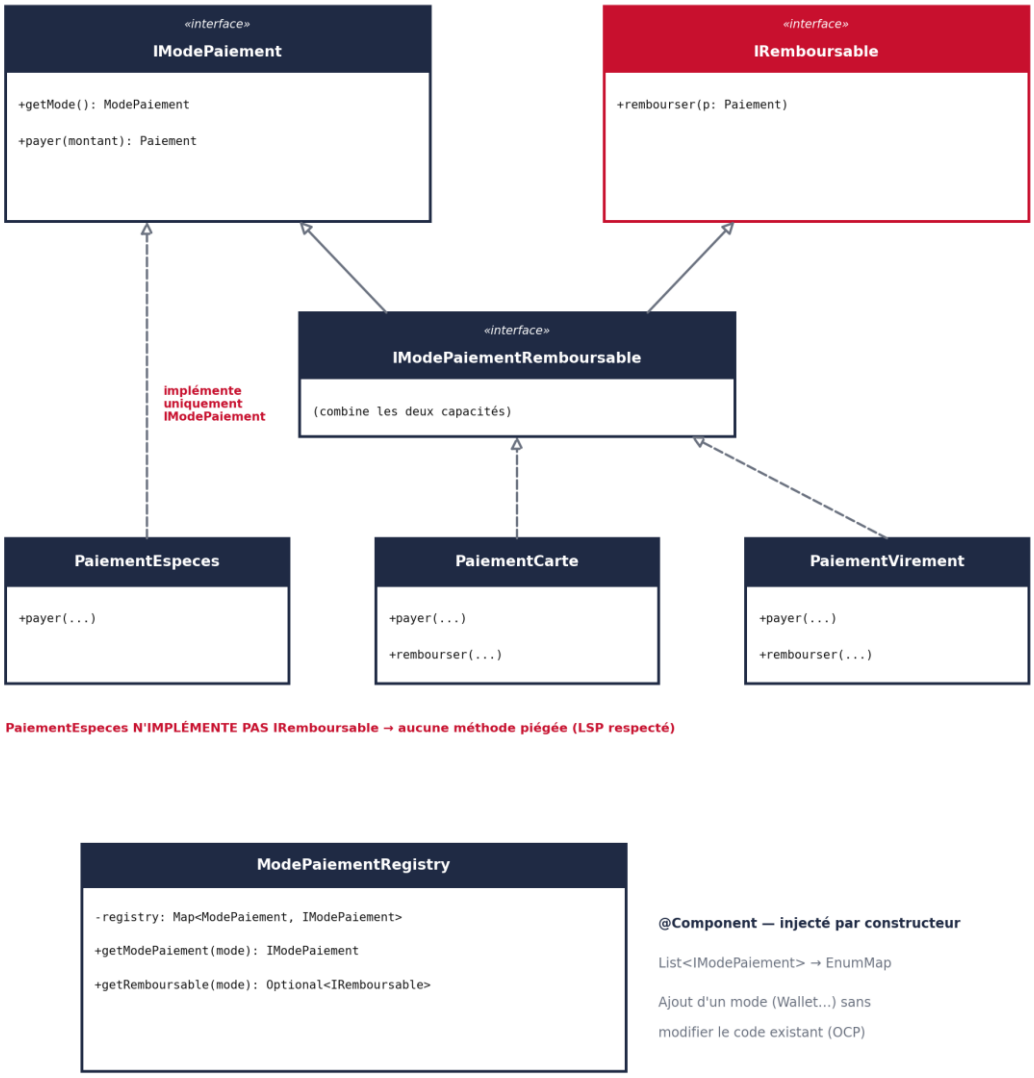
Pour respecter strictement le LSP, nous avons appliqué l'ISP (Interface Segregation Principle). L'opération d'encaissement (universelle) et l'opération de remboursement (réservée aux flux électroniques/bancaires) ont été scindées en deux abstractions distinctes.

Critère de comparaison	Avant (proposition stagiaire)	Après (notre solution Clean Code)
Contrat d'interface	Interface 'grasse' IModePaiement forçant payer() ET rembourser()	Interfaces ségréguées : IModePaiement (payer) et IRemboursable (rembourser)
Comportement d'espèces	PaiementEspeces jette UnsupportedOperationException	PaiementEspeces n'implémente que IModePaiement. Zéro méthode piégée.
Sécurité polymorphique	Polymorphisme piégé : risque d'exception au runtime	Substitutabilité garantie à 100 % : tout appel de méthode est sûr.
Traitement du cash	Crash serveur ou 'instanceof' hardcodé	Gestion métier fluide : génération d'un bon d'achat client et audit sans exception

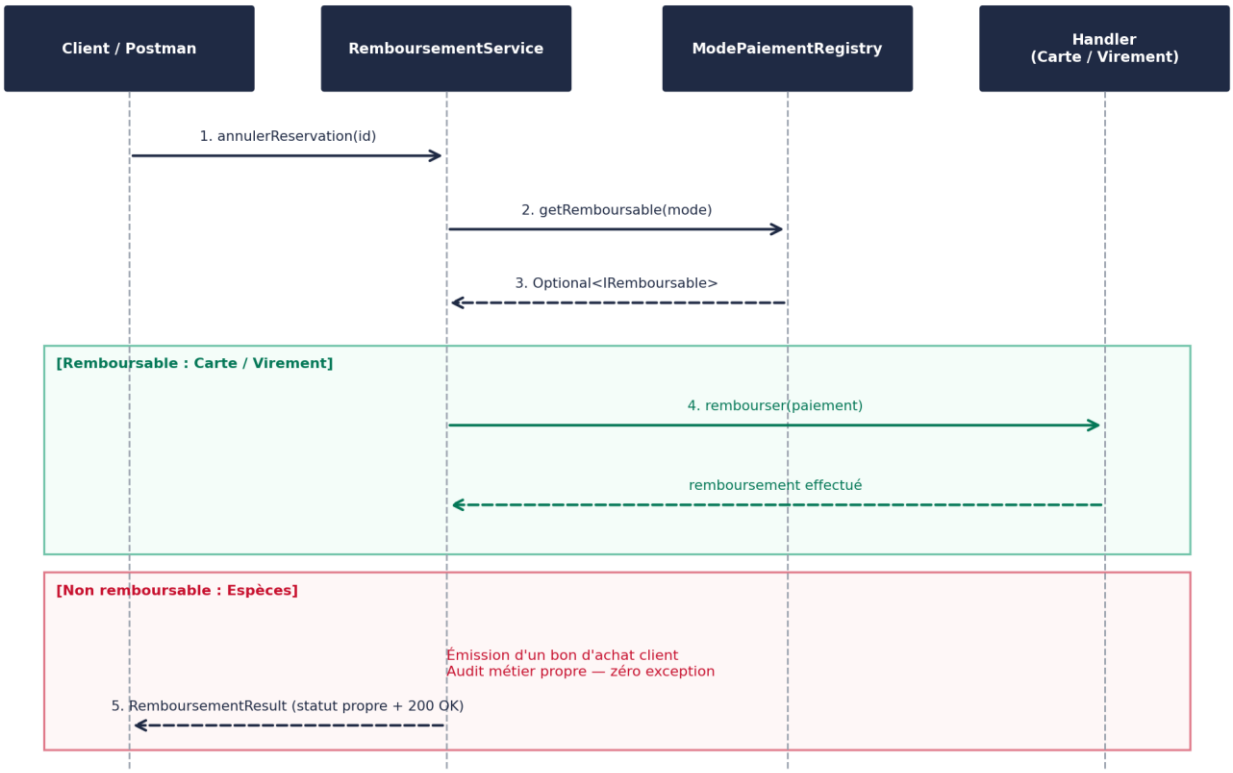
2 Schéma de la solution

L'architecture repose sur la séparation nette des rôles, l'utilisation d'interfaces de capacité et d'un registre dynamique injecté par constructeur Spring (conforme aux exigences de testabilité et SonarQube S6813).

2.1 Diagramme de classes UML de la conception



2.2 Diagramme de séquence du traitement du remboursement



3 Proposition de code détaillée

Voici le code implémenté dans notre projet qui résout définitivement la problématique de la Mission 3.

3.1 Interface de base IModePaiement (ISP)

Emplacement : *src/main/java/tn/esprit/boudourcce11/service/location/payment/IModePaiement.java*

```
package tn.esprit.boudourcce11.service.location.payment;

import java.math.BigDecimal;
import tn.esprit.boudourcce11.domain.ModePaiement;
import tn.esprit.boudourcce11.domain.Paiement;

public interface IModePaiement {
    ModePaiement getMode();
    Paiement payer(BigDecimal montant);
}
```

3.2 Interface de capacité IRemboursable

Emplacement : *src/main/java/tn/esprit/boudourcce11/service/location/payment/IRemboursable.java*

```
package tn.esprit.boudourcce11.service.location.payment;

import tn.esprit.boudourcce11.domain.Paiement;

public interface IRemboursable {
    void rembourser(Paiement paiement);
}
```

3.3 Interface combinée IModePaiementRemboursable

Emplacement : *src/main/java/tn/esprit/boudourcce11/service/location/payment/IModePaiementRemboursable.java*

```
package tn.esprit.boudourcce11.service.location.payment;

public interface IModePaiementRemboursable extends IModePaiement, IRemboursable {
}
```

3.4 Implémentation PaiementEspeces (conforme LSP — aucune exception jetée)

Emplacement : *src/main/java/tn/esprit/boudourcce11/service/location/payment/PaiementEspeces.java*

```java
package tn.esprit.boudourcce11.service.location.payment;

import java.math.BigDecimal;
import java.time.LocalDate;
import org.springframework.stereotype.Component;
import tn.esprit.boudourcce11.domain.ModePaiement;
import tn.esprit.boudourcce11.domain.Paiement;

@Component
public class PaiementEspeces implements IModePaiement {
    @Override
    public ModePaiement getMode() {
        return ModePaiement.ESPECES;
    }

    @Override
    public Paiement payer(BigDecimal montant) {
        System.out.println("[ESPECES] Encaissement en especes effectue a l'agence.");
        return Paiement.builder()
                .datePaiement(LocalDate.now())
                .modePaiement(ModePaiement.ESPECES)
                .montant(montant)
                .build();
    }
    // N'implémente PAS IRemboursable : zéro méthode inutile ou piégée !
}
```

3.5 Registre centralisé ModePaiementRegistry (respect OCP & DIP)

Emplacement : *src/main/java/tn/esprit/boudourcce11/service/location/payment/ModePaiementRegistry.java*

```java
package tn.esprit.boudourcce11.service.location.payment;

import java.util.*;
import org.springframework.stereotype.Component;
import tn.esprit.boudourcce11.domain.ModePaiement;

@Component
public class ModePaiementRegistry {
    private final Map<ModePaiement, IModePaiement> registry = new EnumMap<>(ModePaiement.class);

    public ModePaiementRegistry(List<IModePaiement> modes) {
        for (IModePaiement mode : modes) {
            if (mode.getMode() != null) {
                registry.put(mode.getMode(), mode);
            }
        }
    }

    public IModePaiement getModePaiement(ModePaiement mode) {
        IModePaiement handler = registry.get(mode);
        if (handler == null) throw new IllegalArgumentException("Mode non supporte : " + mode);
        return handler;
    }

    public Optional<IRemboursable> getRemboursable(ModePaiement mode) {
        IModePaiement handler = registry.get(mode);
        if (handler instanceof IRemboursable remboursable) {
            return Optional.of(remboursable);
        }
        return Optional.empty();
    }
}
```

3.6 Service métier RemboursementServiceImpl

Emplacement : *src/main/java/tn/esprit/boudourcce11/service/location/payment/RemboursementServiceImpl.java*

```java
@Service
@RequiredArgsConstructor
public class RemboursementServiceImpl implements IRemboursementService {
    private final ModePaiementRegistry modePaiementRegistry;
    private final IReservationRepository reservationRepository;
    private final IContratRepository contratRepository;

    @Override
    public RemboursementResult traiterRemboursement(Paiement paiement) {
        if (paiement == null) return RemboursementResult.aucunPaiement();

        Optional<IRemboursable> remboursableOpt =
                modePaiementRegistry.getRemboursable(paiement.getModePaiement());
        if (remboursableOpt.isPresent()) {
            remboursableOpt.get().rembourser(paiement);
            return RemboursementResult.succes(paiement.getMontant(), paiement.getModePaiement());
        } else {
            // Cas des ESPECES : gestion propre SANS exception technique
            System.out.println("[AUDIT] Remboursement cash impossible a distance. Bon d'achat genere.");
            return RemboursementResult.nonRemboursableEspeces();
        }
    }
}
```

4 Prompts de la phase de découverte

Prompt (tel quel)	Ce que nous en avons retenu	Ce que nous avons corrigé ou écarté, et pourquoi
1. *« Dans un système de location, un stagiaire a écrit une interface IModePaiement avec payer() et rembourser(). Pour PaiementEspeces, il lance throw new UnsupportedOperationException("impossible"). Quel principe SOLID est violé et pourquoi ? »*	Le principe violé est le LSP (Liskov Substitution Principle). Un sous-type ne doit jamais refuser une méthode déclarée par son abstraction parente ni lever d'exception inattendue qui casserait le polymorphisme du code appelant.	L'IA suggérait d'attraper l'exception dans un bloc try/catch au niveau du contrôleur. Nous avons totalement **ÉCARTÉ** cette suggestion car utiliser des exceptions pour gérer un cas métier prévisible est un anti-pattern notoire.
2. *« Comment corriger cette violation de Liskov sans forcer le code client à faire des tests 'if (mode instanceof PaiementEspeces)' ? »*	La solution élégante consiste à combiner LSP avec l'ISP : séparer la capacité d'encaisser (IModePaiement) de la capacité de rembourser (IRemboursable). Seuls les modes supportant le remboursement implémentent cette capacité.	L'IA proposait d'ajouter un booléen 'boolean estRemboursable()' dans IModePaiement. Nous avons **ÉCARTÉ** cette piste car elle laissait une méthode rembourser() vide dans PaiementEspeces. L'interface de capacité est bien plus propre et typée.
3. *« Comment intégrer cette architecture dans une application Spring Boot respectant l'injection par constructeur et la règle SonarQube S6813 ? »*	Utiliser un composant ModePaiementRegistry injecté par constructeur (@RequiredArgsConstructor) avec List<IModePaiement>, qui mappe dynamiquement les stratégies sans couplage fort.	L'IA proposait au départ d'injecter chaque bean avec @Qualifier("especes") directement dans le service. Nous l'avons **CORRIGÉ** en passant par un Registry car l'injection fixe aurait violé le principe OCP lors de l'ajout futur d'autres modes (ex : Wallet).

Synthèse de la démarche

Notre travail a démontré qu'une IA peut identifier le nom d'un principe, mais que seul l'esprit critique du concepteur logiciel permet d'éliminer les mauvaises solutions (try/catch d'exception métier, instanceof) au profit d'une architecture extensible et conforme aux règles professionnelles (LSP + ISP + SonarQube S6813).

Youssef El Harizi · Boudour Zlaoui · Khouloud Fares

4CCE11